OpenClaw AI Script Peripheral Control Tutorial

> Before starting, please complete the peripheral hardware configuration, API Key configuration, model configuration, and three dialogue solutions configuration tests under the "Pre-usage Preparation" directory. If you are currently only using TUI/WhatsApp, you can skip the voice interaction configuration for now. This article assumes the Jetson Nano Docker environment is already prepared and only expands on "Having AI generate independently runnable scripts".

1. Tutorial Objectives

This article demonstrates that OpenClaw on Jetson Nano can not only directly control peripherals but also save a set of control logic as Python script files through "natural language generation of hardware scripts". After completing this tutorial, readers will be able to:

- Understand the basic approach for OpenClaw on Jetson to generate hardware control scripts
- Generate peripheral scripts through WhatsApp, TUI, and voice three ways
- Generate common scripts for RGB, servos, cameras, temperature and humidity, OLED, music playback, etc.
- Generate comprehensive scripts that combine multiple peripherals

2. Hardware Preparation

- Jetson Nano B01
- OpenClaw Expansion Board
- `[Optional] RGB Light Strip / Servo / CSI Camera`
- `[Optional] AI Voice Interaction Module + Speaker`

Note:

- This tutorial does not require all peripherals to be connected at once.
- Prepare the corresponding hardware based on which type of script you want to demonstrate.
- Peripherals such as camera, DHT, OLED, etc. should first complete individual control tests before doing combined scripts.

3. Software Preparation

Enter the Jetson Docker container:

```
~/run_usb_docker.sh
```

Common actions are organized into commands like the following:

```
/usr/local/bin/dht --format json   # Get sensor data
/usr/local/bin/dht_oled --format json  # Display sensor data on OLED
/usr/local/bin/rgb --format json color --color red # Turn all lights red
/usr/local/bin/rgb --format json effect --effect breathe --color blue --speed 5 # Blue
breathing light
/usr/local/bin/servo call-tool Servo1 --action set_angle --angle 90  # Servo 1 rotates to 90
degrees
/usr/local/bin/oled text --text 'Hello OpenClaw'
/usr/local/bin/oled clear
/usr/local/bin/csi_snap snap
/usr/local/bin/csi_snap snap --width 1280 --height 720
```

The core of the current script generation capability is not "execute a single immediate command", but having OpenClaw generate an independently runnable Python script based on natural language requirements. You can think of it as two usage modes:

- Mode 1: Only generate and save the script, view or run it yourself later.
- Mode 2: Generate the script and run it immediately to directly observe peripheral effects.

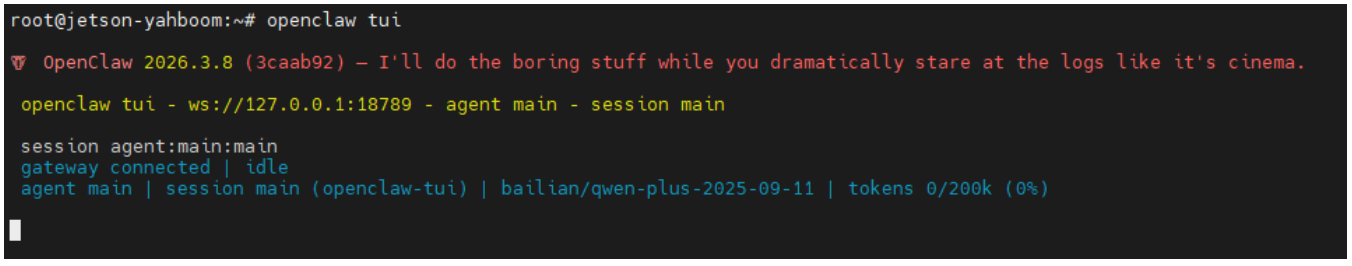
If you only say "generate script", the system should only save the script and should not automatically trigger hardware actions; if you say "generate script and run", "run it", "execute it", the system will run the script after saving.

4. Testing Script Generation Capability

Before starting this tutorial, please complete one `openclaw tui` basic conversation self-check. Confirm that OpenClaw can respond normally before entering the script generation specific test.

Enter TUI with the following command:

```
openclaw tui
```



It is recommended to start with the simplest script requirements, for example:

- Write a script to display current temperature and humidity on OLED
- Write a script that makes RGB blue breathing light last for 5 seconds
- Write a script to rotate servo 1 to 180 degrees

Under normal circumstances, the response should include:

- The generated script path, for example `/root/.openclaw/workspace/scripts/xxx.py`
- The run command, for example `python3 /root/.openclaw/workspace/scripts/xxx.py`

5. Three Methods for Communicating with OpenClaw

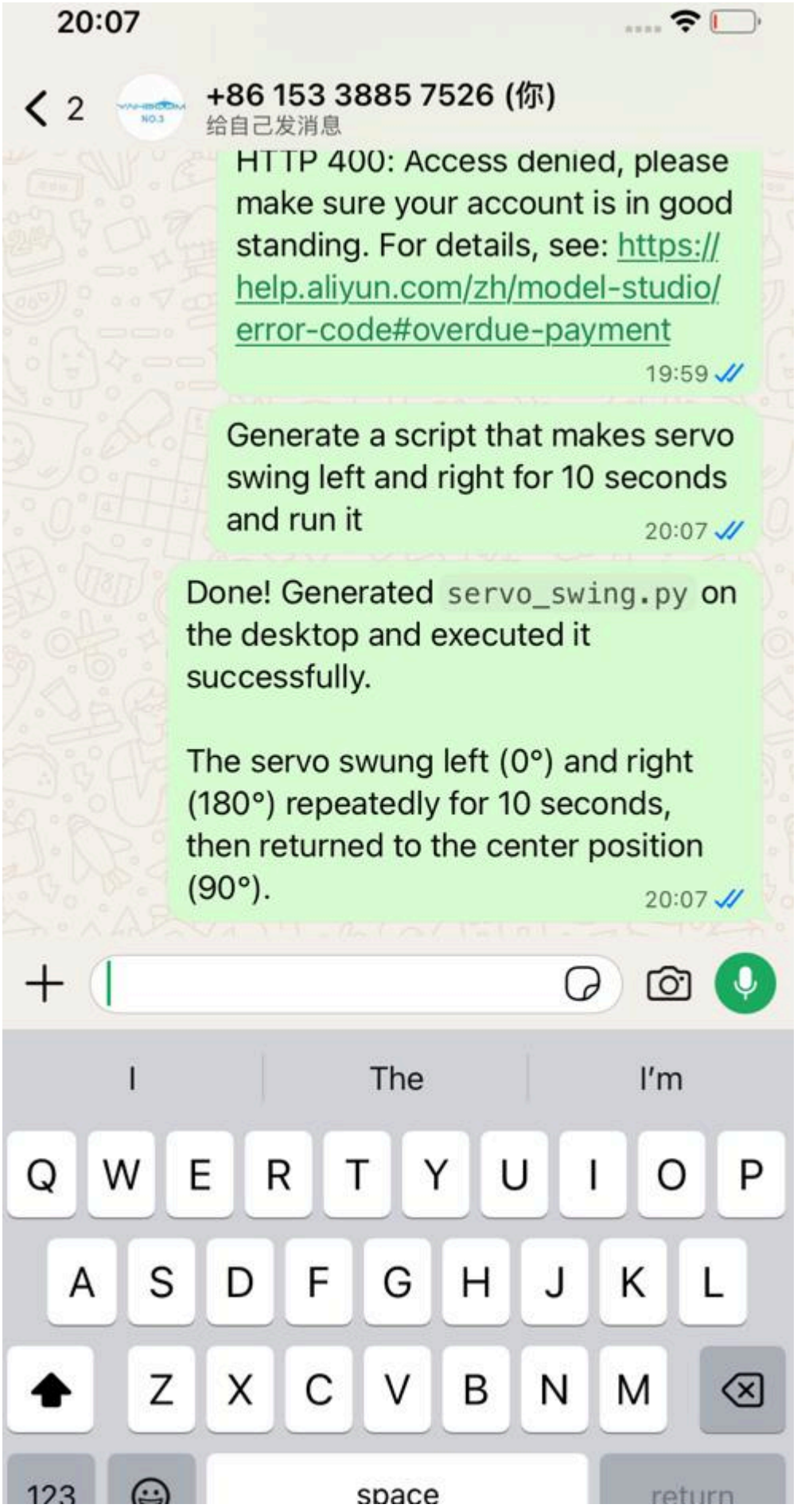
Method	Suitable Scenario	Advantages	Suggested Positioning
WhatsApp	Remote sending requests, classroom demonstration	Can directly describe "write what script for me"	Demonstrate natural language script generation
TUI	Local debugging, verify generation results	Easiest to see script paths, run output and error messages	Preferred debugging solution
Voice Code	Voice interaction, complete AI experience	Can verbally describe script requirements	Demonstrate complete voice closed-loop

5.1 WhatsApp Solution

For WhatsApp access steps, please refer to [05-Three Dialogue Solution Configuration Tests], this section only retains dialogue examples suitable for script generation tutorial.

You can directly send:

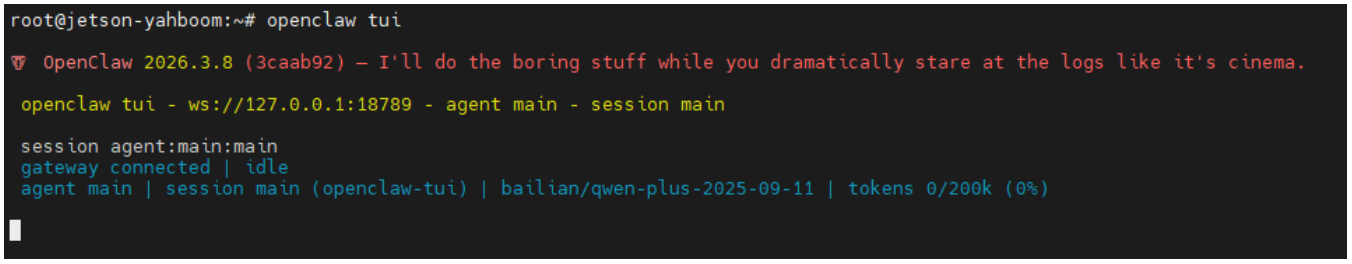
- Write a script to get current temperature and humidity and display on OLED
- Generate a script that make servo swing left and right for 10 seconds and run it
- Write a script to control RGB to light up red then turn off, loop 5 times


20:07
2
+86 153 3885 7526 (你)
给自己发消息
HTTP 400: Access denied, please make sure your account is in good standing. For details, see: https://help.aliyun.com/zh/model-studio/error-code#overdue-payment
19:59
Generate a script that makes servo swing left and right for 10 seconds and run it
20:07
Done! Generated servo_swing.py on the desktop and executed it successfully.
The servo swung left (0°) and right (180°) repeatedly for 10 seconds, then returned to the center position (90°).
20:07
I
The
I'm
Q W E R T Y U I O P
A S D F G H J K L
Z X C V B N M
123
space
return

5.2 TUI Solution

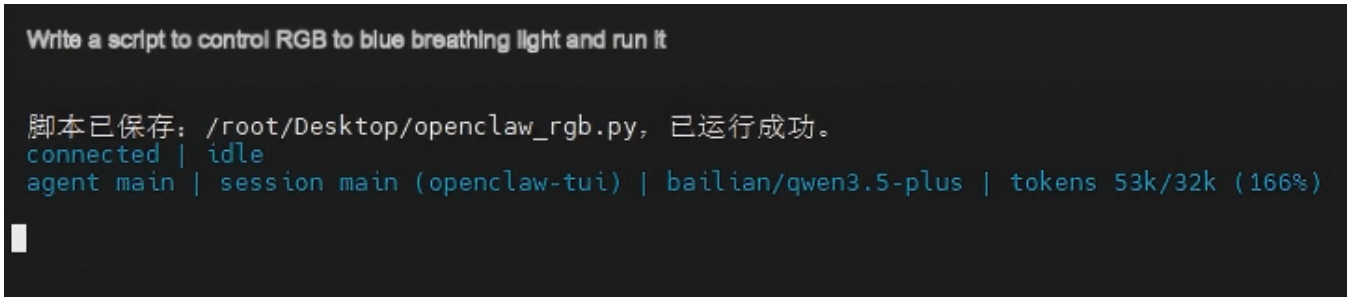
If basic configuration is complete, directly enter in the container:

```
openclaw tui
```



Then input script requirements, for example:

- Write a script to control RGB to blue breathing light and run it



Desktop > openclaw_rgb.py

```python
#!/usr/bin/env python3
"""Generated OpenClaw hardware script.

This script uses the approved Jetson wrapper only:
    /usr/local/bin/openclaw_hw
"""
from __future__ import annotations

import json
import subprocess

OPENCLAW_HW = "/usr/local/bin/openclaw_hw"
STEPS = [
    {
        "label": "RGB blue breathe",
        "args": [
            "rgb-breathe",
            "blue"
        ]
    }
]


def run_hw(*args: str) -> dict:
    completed = subprocess.run(
        [OPENCLAW_HW, *map(str, args)],
        check=False,
        text=True,
        capture_output=True,
        timeout=30,
    )
    print("$", OPENCLAW_HW, *map(str, args))
    if completed.stdout:
        print(completed.stdout.strip())
```

TUI is the most recommended debugging method. Because it can directly see model responses, script paths, execution output, and error information.

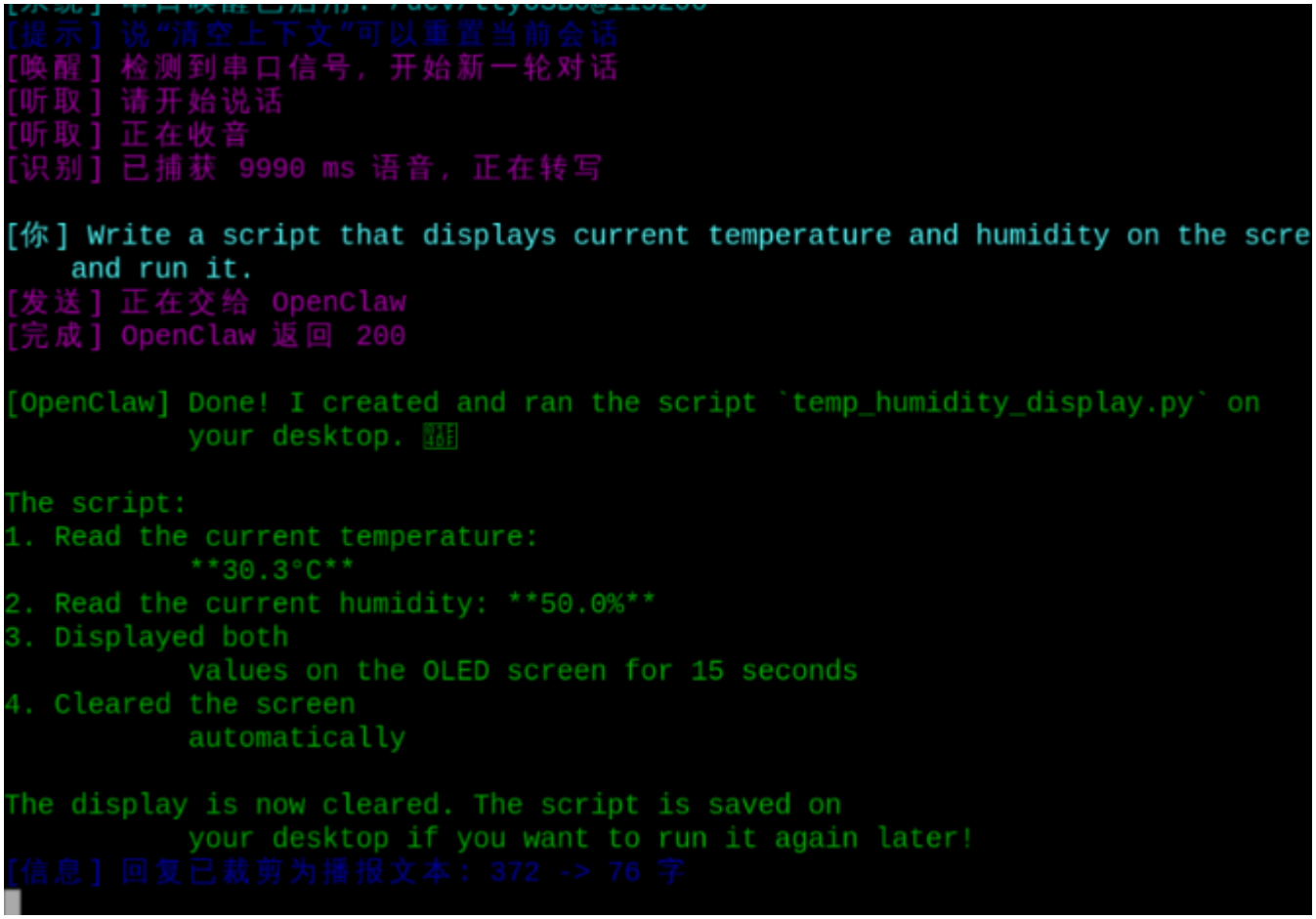
5.3 Voice Code Solution

```
Requires optional AI Voice Interaction Module + Speaker
```

For voice module installation, environment configuration and basic wake-up process, please refer to [04-AI Voice Interaction Configuration]. After waking up, you can directly say:

- Write a script that displays current temperature and humidity to the screen and run it
- Write a script to make servo 1 rotate to 180 degrees
- Write a script to make RGB light up red and run

Voice solution recommends using short sentences. Too long combined requirements can be split into two steps: first have OpenClaw generate the script, then have it run the script.



5.4 Closing Loop Running Scripts

For example:

After sending the command `write a script to get temperature and humidity every 5 seconds and display on OLED and execute`, the program keeps running in the background. The closing method is as follows:

First find the corresponding file in `/root/.openclaw/workspace/scripts` directory, for example the file I generated is dht_oled_loop.py, then input command to query the task's PID and close the process:

```
root@jetson-yahboom:~# pgrep -f dht_oled_loop.py
23208
root@jetson-yahboom:~# kill 23208
```

6. Comprehensive Cases

The following cases default to demonstration in `openclaw tui`. WhatsApp and voice solutions can also use similar expressions.

6.1 Case 1: RGB Light Effect Script

Experimental Objective

Have OpenClaw generate a basic lighting script to verify the "script creation + peripheral execution" chain.

Example Commands

- Write a script to make RGB red light blink 5 times
- Generate a python file to control RGB as blue breathing light
- Write a script to make RGB do yellow flowing water light effect

6.2 Case 2: Servo Action Script

Experimental Objective

Generate a servo action script to solidify a single conversation action into a repeatable program.

Example Commands

- Write a script to make servo 1 move back and forth between 0, 90, and 180 degrees
- Generate a servo nod script and run it

6.3 Case 3: Temperature/Humidity and OLED Script

Experimental Objective

Generate a script to read current temperature and humidity and display the results on OLED.

Example Commands

- Generate a script to read temperature and humidity
- Write a script to display current temperature and humidity on OLED

6.4 Case 4: Camera Script

Experimental Objective

Generate a script to call CSI camera to take photos or analyze the frame content.

Example Commands

- Write a script to take a photo
- Write a script to first take a photo, then return the photo path
- Generate a script to take a photo and then describe the frame content

6.5 Case 5: Combined Script

Experimental Objective

Have OpenClaw generate a comprehensive script containing multiple peripheral actions at once.

Example Commands

- Generate a script: first light up green light, then rotate servo to 90 degrees, finally take a photo
- Write a comprehensive script: read temperature and humidity, if temperature is too high light up red light, and display "Too Hot" on OLED

7. Common Problems

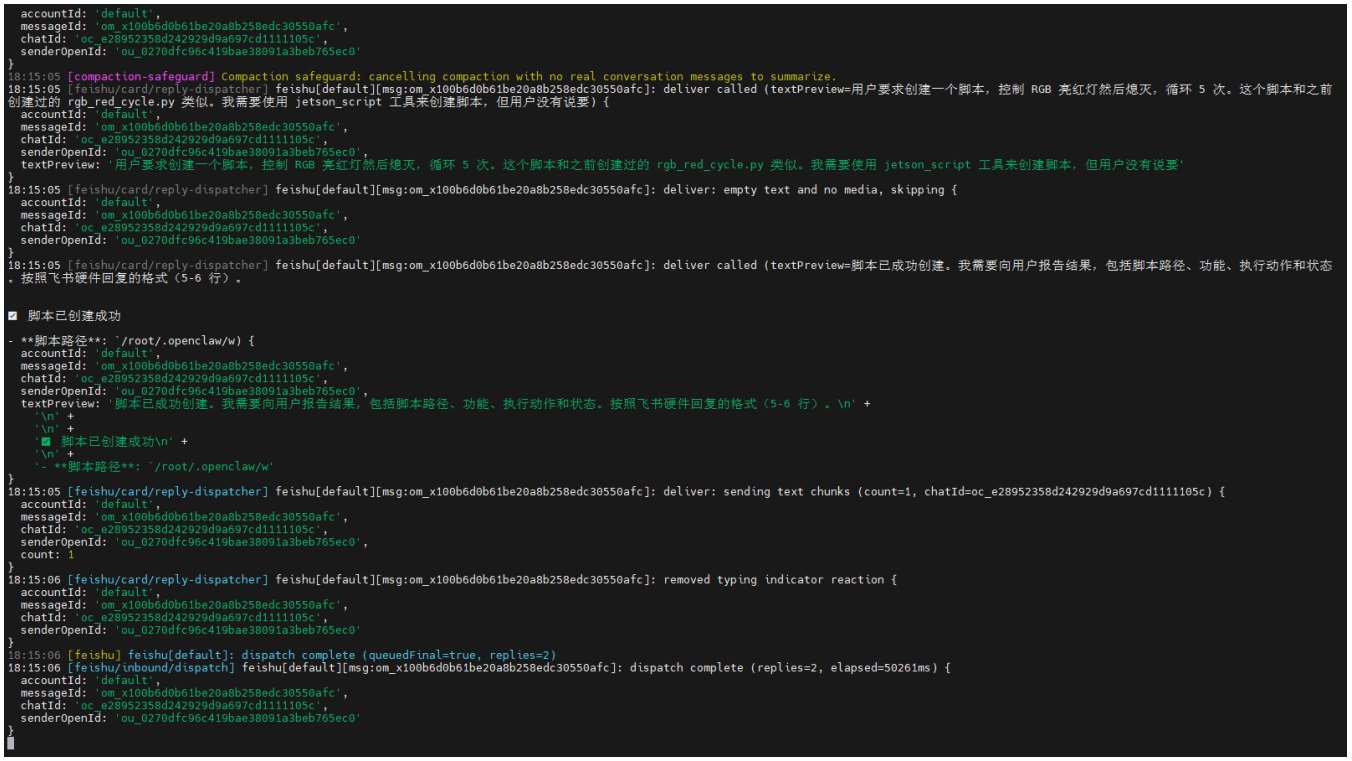
7.1 Response includes script content but no file

This usually means the Jetson-side script generation chain was not truly entered. The correct result should include a path like:

```
/root/.openclaw/workspace/scripts/xxx.py
```

7.2 Script ran but peripheral has no action

Check the gateway log, whether there is a log similar to the following:



7.3 What is the difference between generating script and immediately controlling peripheral

- "Make RGB light up red": leans towards immediately executing a hardware action once.
- "Write a script to make RGB light up red": leans towards generating a reusable script.
- "Write a script to make RGB light up red and run": first generate script, then execute script.